

Project Documentation

AWS Dynamic Inventory Server Monitoring using Ansible

Project Overview

This project automates monitoring of AWS EC2 instances using Ansible Dynamic Inventory.

The solution:

- Automatically discovers running EC2 instances
- Connects using PEM authentication
- Collects system health information
- Generates centralized HTML reports
- Sends monitoring reports through email

This project demonstrates practical DevOps automation and cloud infrastructure monitoring.

Architecture

Infrastructure Used

Component	Purpose
AWS EC2	Servers
Ansible Control Node	Automation server
Worker Nodes	Managed servers
AWS Dynamic Inventory	Auto-discover EC2
Gmail SMTP	Send reports
Linux	Operating system

Project Objectives

- ✓ Automate server monitoring
- ✓ Avoid static inventory management

- ✓ Dynamically discover EC2 instances
 - ✓ Collect CPU, Memory, Disk, and Time info
 - ✓ Generate centralized HTML reports
 - ✓ Send reports automatically via email
-

Project Structure

/etc/ansible/

```
|— ansible.cfg
|— aws_ec2.yml
|— dynamic.pem
|— playbooks/
|   |— info.yml
```

✓ STEP 1 — EC2 Setup

Created:

- 1 Control Node
- 2 Worker Nodes

Requirements:

- Same key pair for all instances
 - Security Group allows SSH (Port 22)
-

✓ STEP 2 — Install Ansible on Control Node

```
sudo yum update -y
sudo yum install ansible -y
```

Verify:

```
ansible --version
```

✓ STEP 3 — Install AWS Dependencies

```
sudo yum install python3-pip -y
pip3 install boto3 botocore
ansible-galaxy collection install amazon.aws
```



STEP 4 — Configure AWS Credentials

`aws configure`

Provide:

- AWS Access Key
 - AWS Secret Key
 - Region: ap-south-1
-



STEP 5 — Configure PEM File

Move PEM file:

```
sudo mv dynamic.pem /etc/ansible/
```

Set permissions:

```
sudo chown ec2-user:ec2-user /etc/ansible/dynamic.pem
sudo chmod 400 /etc/ansible/dynamic.pem
```



STEP 6 — Go to Ansible Directory

```
cd /etc/ansible
```



STEP 7 — Create AWS Dynamic Inventory File

Create file:

```
sudo vi aws_ec2.yml
```

Paste:

```
plugin: amazon.aws.aws_ec2

regions:
```

```
- ap-south-1

hostnames:
  - private-ip-address

filters:
  instance-state-name: running

compose:
  ansible_host: private_ip_address
  ansible_user: ec2-user
  ansible_ssh_private_key_file: /etc/ansible/dynamic.pem

keyed_groups:
  - key: tags.Name
    prefix: ""
```

STEP 8 — Configure Ansible

Create config file:

```
sudo vi ansible.cfg
```

Paste:

```
[defaults]
inventory = aws_ec2.yml
private_key_file = /etc/ansible/dynamic.pem
host_key_checking = False

[inventory]
enable_plugins = amazon.aws.aws_ec2
```

STEP 9 — Verify Dynamic Inventory

Run:

```
ansible-inventory --graph
```

Expected output:

```
@all:
  |--@ungrouped:
  |   |--Work_Node_1
  |   |--Work_Node_2
  |   |--Control_Node
```

STEP 10 — Test Connectivity

Single node:

```
ansible -i aws_ec2.yml Work_Node_1 -m ping
```

All worker nodes:

```
ansible -i aws_ec2.yml Work_Node_* -m ping
```

Expected:

```
SUCCESS => {
    "ping": "pong"
}
```



STEP 11 — Install Mail Dependency

```
sudo yum install python3-pip -y
pip3 install secure-smtplib
```



STEP 12 — Gmail SMTP Configuration

Requirements:

- Enable 2-Factor Authentication
- Generate Gmail App Password
- <https://myaccount.google.com/apppasswords>

Use App Password in playbook.



STEP 13 — Create Monitoring Playbook

Create directory:

```
mkdir -p playbooks
```

Create playbook:

```
vi playbooks/info.yml
```

Paste complete playbook:

```

---
- name: AWS Dynamic Inventory Server Monitoring
  hosts: _W_server_*
  become: yes
  gather_facts: yes

  tasks:

    - name: Remove old HTML report files
      shell: rm -f /tmp/*.html
      delegate_to: localhost
      run_once: true

    - name: Get Memory Information
      shell: free -h
      register: memory_info

    - name: Get CPU Load Information
      shell: uptime
      register: cpu_info

    - name: Get Disk Usage Information
      shell: df -h /
      register: disk_info

    - name: Get Current Server Time
      shell: date
      register: time_info

    - name: Create HTML row for each server
      copy:
        dest: "/tmp/{{ inventory_hostname }}.html"
        content: |
          <tr>

            <td>{{ hostvars[inventory_hostname]['tags']['Name'] |
default(inventory_hostname) }}</td>

            <td>
              <pre>{{ memory_info.stdout }}</pre>
            </td>

            <td>
              <pre>{{ cpu_info.stdout }}</pre>
            </td>

            <td>
              <pre>{{ disk_info.stdout }}</pre>
            </td>

            <td>
              {{ time_info.stdout }}
            </td>

          </tr>

      delegate_to: localhost

    - name: Generate Final HTML Report
      hosts: localhost
      gather_facts: no

```

```
vars:
  recipient_email: "yourmail@gmail.com"

tasks:

- name: Combine all HTML rows
  assemble:
    src: /tmp
    dest: /tmp/final_server_rows.html
    regexp: '.*\.html$'

- name: Create Final HTML Report
  copy:
    dest: /tmp/aws_health_report.html
    content: |
      <html>

      <head>

        <style>

          body {
            font-family: Arial, sans-serif;
          }

          table {
            border-collapse: collapse;
            width: 100%;
          }

          th, td {
            border: 1px solid #dddddd;
            padding: 10px;
            text-align: left;
            vertical-align: top;
          }

          th {
            background-color: #f2f2f2;
          }

          h2 {
            color: #333333;
          }

          pre {
            margin: 0;
            font-size: 12px;
          }

        </style>

      </head>

      <body>

        <h2>AWS Dynamic Inventory Server Health Report</h2>

        <table>
```

```

        <tr>
          <th>Server Name</th>
          <th>Memory</th>
          <th>CPU</th>
          <th>Disk</th>
          <th>Time</th>
        </tr>

        {{ lookup('file', '/tmp/final_server_rows.html') }}

      </table>

    </body>

  </html>

- name: Send Email Report
  mail:
    host: smtp.gmail.com
    port: 587
    username: "yourmail@gmail.com"
    password: "YOUR_APP_PASSWORD"
    to: "{{ recipient_email }}"
    subject: "AWS Dynamic Inventory Server Health Report"
    subtype: html
    body: "{{ lookup('file', '/tmp/aws_health_report.html') }}"
    secure: starttls

```

✓ STEP 14 — Validate Playbook Syntax

```
ansible-playbook --syntax-check playbooks/info.yml
```

Expected:

```
playbook: playbooks/info.yml
```

✓ STEP 15 — Execute Playbook

```
ansible-playbook -i aws_ec2.yml playbooks/info.yml
```

✉ Final Output

Email contains:

Server Name	Memory	CPU	Disk	Time
Work_Node_1	RAM Info	CPU Load	Disk Usage	Current Time
Work_Node_2	RAM Info	CPU Load	Disk Usage	Current Time

Server Name	Memory	CPU	Disk	Time
Control_Node	RAM Info	CPU Load	Disk Usage	Current Time

Features Implemented

- ✓ AWS Dynamic Inventory
 - ✓ EC2 Auto Discovery
 - ✓ SSH Automation
 - ✓ Linux Monitoring
 - ✓ CPU Monitoring
 - ✓ Memory Monitoring
 - ✓ Disk Monitoring
 - ✓ HTML Report Generation
 - ✓ Email Automation
 - ✓ Multi-Server Monitoring
 - ✓ Infrastructure Automation
-

Problems Faced During Project

Problem	Solution
Dynamic inventory not detecting hosts	Fixed AWS plugin configuration
SSH connection failure	Corrected PEM permissions
Duplicate email reports	Deleted old /tmp/*.html files
YAML syntax errors	Fixed Jinja indentation
Gmail authentication failed	Used Gmail App Password

Skills Demonstrated

- Ansible
- AWS EC2
- Dynamic Inventory
- Linux Administration
- Infrastructure Automation
- YAML
- SMTP Configuration
- Cloud Monitoring
- DevOps Practices

Resume Description

Built an automated AWS EC2 monitoring solution using Ansible Dynamic Inventory to collect CPU, memory, disk, and server time information from multiple Linux servers and generate centralized HTML email health reports.